DEFENSIVE CYBERSECURITY PORTFOLIO PROJECT

Kubernetes Container Security and Runtime Detection Lab

Final Project Report

PLATFORM **Kubernetes / Minikube**	OPERATING SYSTEM **Ubuntu 24.04 LTS**
RUNTIME SECURITY **Falco**	CONTAINER RUNTIME **containerd**
PROJECT TYPE **Defensive Cybersecurity Lab**	DATE **August 2026**

Deploy -> Identify -> Test -> Detect -> Harden -> Retest -> Document

Repository: github.com/osama-git05/kubernetes-container-security-lab

Table of Contents

Final Project Report

Project Overview

1. Project Objectives

2. Project Scope

3. Lab Architecture

4. Environment Setup

5. Nginx Workload Deployment

6. Insecure Kubernetes Workload

7. Falco Runtime Security Monitoring

8. Controlled Security Testing

9. Custom Falco Detection Rule

10. Kubernetes Workload Hardening

11. Default-Deny NetworkPolicy

12. Final Project Results

13. Key Lessons Learned

14. Project Limitations

15. Future Improvements

16. Project Conclusion

17. Safety and Authorization

18. Final Evidence

Project Overview

Project outcome

The lab demonstrates vulnerable configuration analysis, Falco runtime detection, custom detection engineering, Kubernetes workload hardening, NetworkPolicy configuration, troubleshooting, and evidence-based documentation.

This project demonstrates a practical Kubernetes container-security workflow using a local Minikube environment.

The lab involved deploying an intentionally insecure Kubernetes workload, identifying security weaknesses, performing controlled runtime-security tests, implementing Falco monitoring, creating a custom Falco detection rule, and hardening the workload using Kubernetes security controls.

The project follows the workflow:

Deploy → Identify → Test → Detect → Harden → Retest → Document

1. Project Objectives

The main objective of this project was to build a practical Kubernetes security lab that demonstrates both insecure and hardened container configurations.

The project addressed the following question:

Can suspicious activity inside a local Kubernetes container be detected, and can insecure workloads be hardened using basic Kubernetes security controls?

The specific objectives were to:

- Build and operate a local Kubernetes cluster using Minikube.
- Deploy and expose an Nginx workload.
- Create an intentionally insecure container configuration.
- Identify common container-security weaknesses.
- Deploy Falco for runtime-security monitoring.
- Perform controlled security tests inside the lab.
- Capture and analyze runtime security alerts.
- Harden a Kubernetes workload using securityContext controls.
- Apply a basic NetworkPolicy.
- Create a custom Falco rule to improve detection coverage.
- Retest previously undetected activity.
- Document all configurations, findings, evidence, and results in GitHub.

---

2. Project Scope

The project was intentionally designed as a compact local security lab rather than a production Kubernetes environment.

Included Components

Component	Purpose
Ubuntu 24.04 VM	Linux environment used to host the security lab
Docker	Container platform and Minikube driver
Minikube	Local single-node Kubernetes cluster
kubectl	Kubernetes command-line administration
Helm	Package management used to deploy Falco
Nginx Deployment	Standard Kubernetes workload used for testing
Nginx Service	Exposes the Nginx application
Insecure Ubuntu Pod	Intentionally weak root/privileged workload
Falco	Runtime-security monitoring and alert generation
Hardened Nginx Pod	Demonstrates safer Kubernetes container settings
NetworkPolicy	Demonstrates basic ingress and egress restrictions
Git and GitHub	Version control, documentation, and evidence storage

Out of Scope

The original one-week project intentionally excluded several enterprise-level features in order to keep the lab achievable and focused.

These included:

- Multi-node Kubernetes clusters
- Cloud-hosted Kubernetes
- SIEM integration
- Prometheus and Grafana
- Advanced admission controllers
- Enterprise secret-management systems

- Complex production applications
- Large-scale penetration-testing frameworks

A custom Falco detection rule was later added as an extension to the original project scope.

---

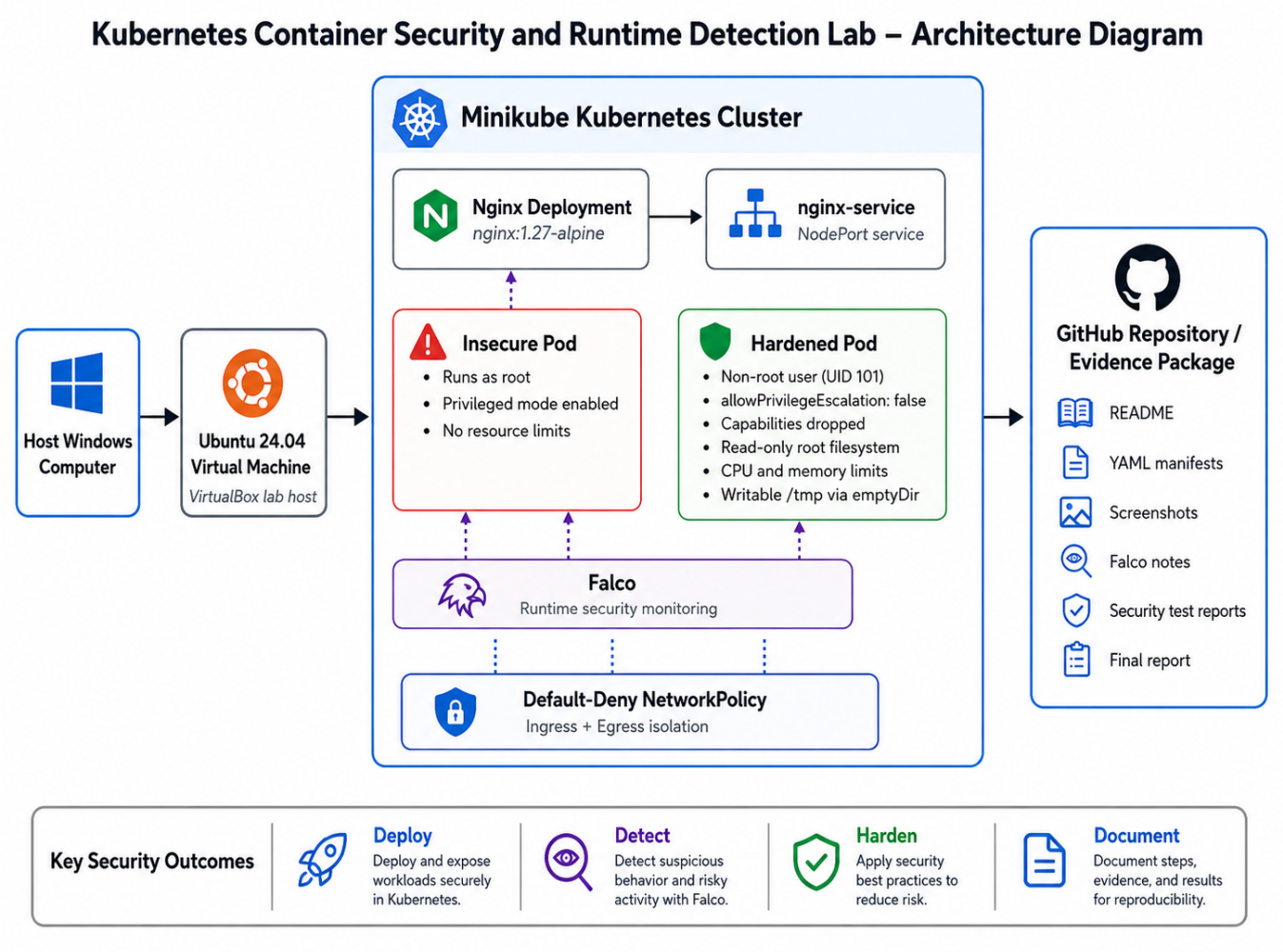
3. Lab Architecture

The project was built using the following logical architecture:

The architecture allows vulnerable and hardened Kubernetes workloads to be tested in the same controlled local environment.

Falco monitors runtime activity inside the Kubernetes cluster, while the NetworkPolicy and hardened security context demonstrate preventative security controls.

Architecture Diagram



Kubernetes Container Security Lab Architecture

4. Environment Setup

The lab was implemented inside an Ubuntu 24.04 LTS virtual machine running in VirtualBox.

The virtual machine was configured with approximately:

- 4 virtual CPU cores
- 8 GB RAM
- 50 GB storage
- Internet connectivity

The following core tools were installed:

- Docker
- kubectl
- Minikube
- Helm
- Git

Docker was verified using:

```
docker run --rm hello-world
```

The successful output confirmed that the Docker installation was functioning correctly.

4.1 Starting the Kubernetes Cluster

Minikube was started using Docker as the driver and containerd as the container runtime:

```
minikube start \
  --driver=docker \
  --container-runtime=containerd \
  --cpus=4 \
  --memory=6144
```

The Kubernetes node status was checked using:

```
kubectl get nodes
```

The Minikube node returned:

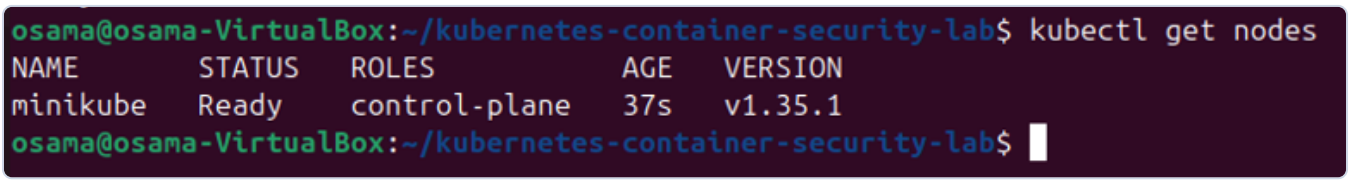
```
Ready
```

Kubernetes system components were then inspected using:

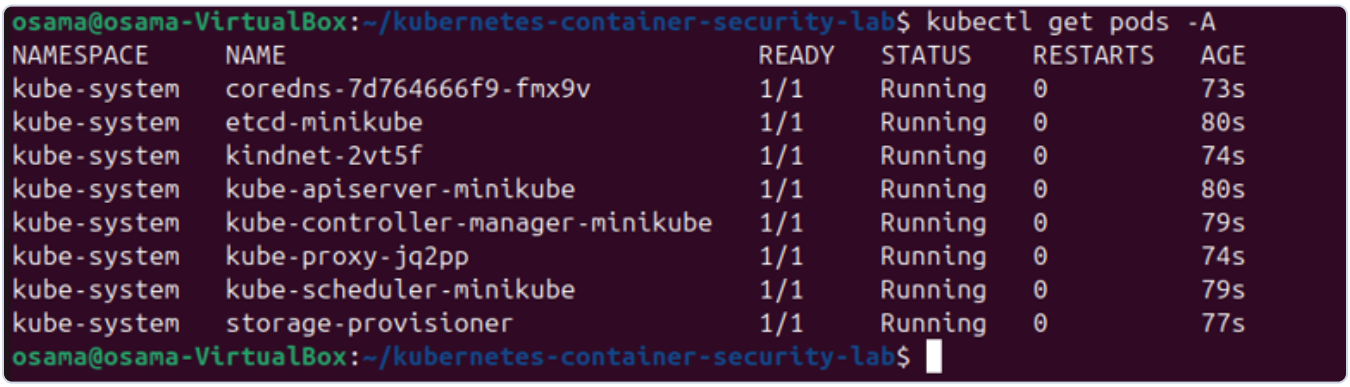
```
kubectl get pods -A
```

The required system pods were running successfully.

Evidence



Cluster Ready



Kubernetes System Pods

This confirmed that the local Kubernetes environment was ready for workload deployment.

---

5. Nginx Workload Deployment

A simple Nginx web application was deployed to provide a normal Kubernetes workload for later security testing.

Two Kubernetes manifests were created:

```
manifests/nginx-deployment.yaml
manifests/nginx-service.yaml
```

5.1 Nginx Deployment

The Deployment used the following container image:

```
nginx:1.27-alpine
```

The manifest created one Nginx replica and exposed container port 80.

The Deployment was applied using:

```
kubectl apply -f manifests/nginx-deployment.yaml
```

5.2 Nginx Service

A NodePort Service was created to expose the Nginx workload.

The Service was applied using:

```
kubectl apply -f manifests/nginx-service.yaml
```

The Kubernetes resources were verified using:

```
kubectl get deployments,pods,services
```

The results confirmed that:

- The Nginx Deployment was available.
- The Nginx pod was running.
- The `nginx-service` existed successfully.

Evidence

```
osama@osama-VirtualBox:~/kubernetes-container-security-lab/manifests$ kubectl get deployments,pods,services
NAME                          READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/nginx-lab     1/1     1            1           35s

NAME                              READY   STATUS    RESTARTS   AGE
pod/nginx-lab-56cc9bdf4d-7ccr4    1/1     Running   0          35s

NAME                    TYPE        CLUSTER-IP      EXTERNAL-IP   PORT(S)        AGE
service/kubernetes      ClusterIP   10.96.0.1       <none>        443/TCP        13m
service/nginx-service   NodePort    10.100.37.47    <none>        80:32076/TCP   5s
osama@osama-VirtualBox:~/kubernetes-container-security-lab/manifests$
```

Nginx Running

5.3 Application Verification

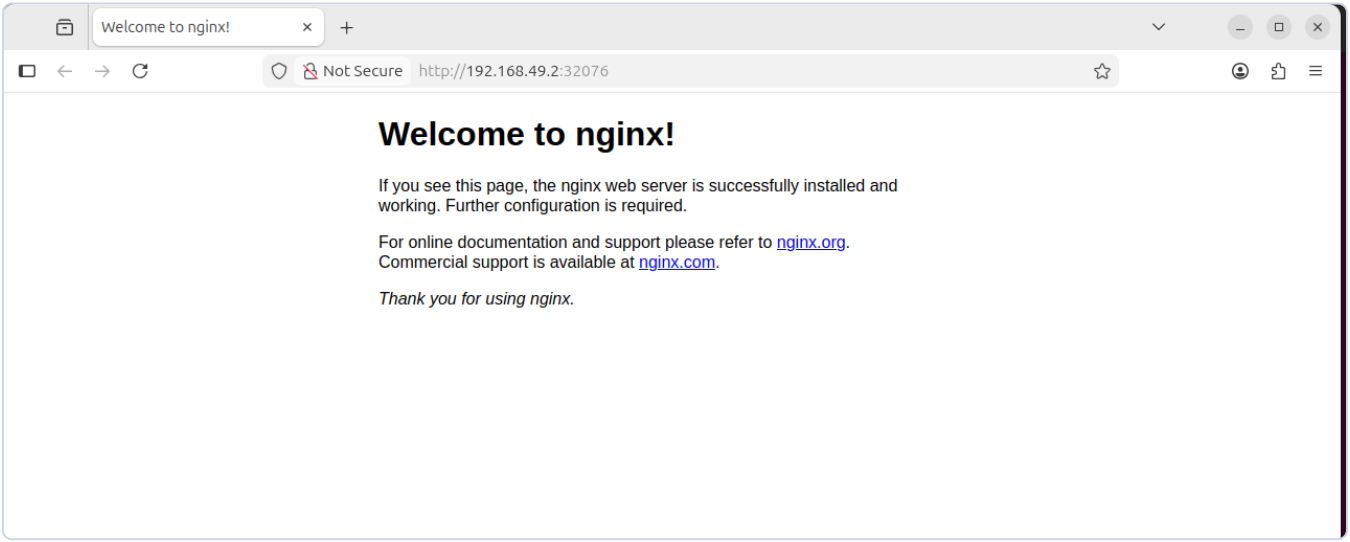
The Nginx service was accessed using:

```
minikube service nginx-service --url
```

The generated Minikube URL was opened in a browser.

The standard Nginx welcome page loaded successfully, confirming that the application was correctly deployed and accessible through Kubernetes.

Evidence



Nginx Website

5.4 Deployment Result

The completion of this stage demonstrated that the Kubernetes cluster could successfully:

- Deploy a containerized application.
- Manage the workload using a Kubernetes Deployment.
- Expose the application through a Service.
- Provide access to the application from the local environment.

This Nginx workload was later used as the target for controlled runtime-security tests and Falco detection.

---

6. Insecure Kubernetes Workload

To establish a security baseline, an intentionally insecure Ubuntu pod was deployed.

The purpose of this workload was to demonstrate several common Kubernetes container-security weaknesses before applying hardening controls.

The manifest was stored in:

```
manifests/insecure-pod.yaml
```

The pod used:

```
ubuntu:24.04
```

and was configured with:

```
securityContext:
  privileged: true
```

The pod was deployed using:

```
kubectl apply -f manifests/insecure-pod.yaml
```

Its status was checked using:

```
kubectl get pod insecure-pod -o wide
```

The pod successfully entered the `Running` state.

6.1 Root User Verification

The container identity was checked using:

```
kubectl exec insecure-pod -- id
```

The result showed:

```
uid=0(root)
```

This confirmed that the container process was running as the root user.

Security Risk

Running containers as root increases the potential impact of a container compromise.

If an attacker gains command execution inside a root container, the attacker may have significantly more control over the container environment than if the application were running as an unprivileged user.

Evidence

```
osama@osama-VirtualBox:~/kubernetes-container-security-lab/manifests$ kubectl exec insecure-pod -- id
uid=0(root) gid=0(root) groups=0(root)
osama@osama-VirtualBox:~/kubernetes-container-security-lab/manifests$
```

Insecure Pod Running as Root

6.2 Privileged Container

The insecure pod was intentionally configured with:

```
privileged: true
```

Privileged mode provides the container with significantly more access to the underlying host environment and weakens normal container isolation.

This configuration should therefore be avoided unless there is a specific operational requirement.

Evidence

```
osama@osama-VirtualBox:~/kubernetes-container-security-lab/manifests$ cat insecure-pod.yaml
apiVersion: v1
kind: Pod
metadata:
  name: insecure-pod
  labels:
    app: insecure-pod
spec:
  containers:
    - name: insecure-container
      image: ubuntu:24.04
      command: ["sleep", "3600"]
      securityContext:
        privileged: true
osama@osama-VirtualBox:~/kubernetes-container-security-lab/manifests$
```

Privileged Security Context

6.3 Missing Resource Limits

The insecure pod did not define:

```
resources:
```

with CPU or memory requests and limits.

Without resource limits, a workload may consume excessive CPU or memory and potentially affect other workloads running on the same Kubernetes node.

6.4 No NetworkPolicy

At this stage of the project, no Kubernetes NetworkPolicy had been applied.

This was verified using:

```
kubectl get networkpolicy
```

No policies were initially present.

This meant the project had not yet introduced Kubernetes network restrictions between workloads.

A default-deny NetworkPolicy was later added during the hardening phase.

6.5 Identified Security Findings

The insecure workload produced four primary findings:

Finding	Evidence	Security Risk
Container runs as root	`uid=0(root)`	Increased impact if the container is compromised
Privileged mode enabled	`privileged: true`	Weakens container isolation and exposes powerful host access
No resource limits	No CPU/memory controls in YAML	Potential excessive resource consumption
No NetworkPolicy	No policy initially present	No project-level Kubernetes network isolation

These findings created the baseline that was later compared with the hardened Kubernetes workload.

6.6 Insecure Baseline Result

The intentionally insecure pod successfully demonstrated several container-security weaknesses.

The workload:

- Ran as root.
- Used privileged mode.
- Had no CPU or memory limits.
- Had no NetworkPolicy protection at that stage.

These weaknesses were intentionally introduced only inside the controlled local lab environment and were documented so that their risks could later be compared against the hardened configuration.

7. Falco Runtime Security Monitoring

Falco was deployed to provide runtime-security monitoring for activity occurring inside Kubernetes containers.

The project guide requires Falco to run successfully and capture at least one readable container-related warning with process and Kubernetes context.

7.1 Installing Falco

The official Falco Helm repository was added using:

```
helm repo add falcosecurity https://falcosecurity.github.io/charts
helm repo update
```

Falco was installed into its own Kubernetes namespace:

```
helm install --replace falco \
  --namespace falco \
  --create-namespace \
  --set tty=true \
  falcosecurity/falco
```

The deployment was allowed time to become ready using:

```
kubectl wait pods \
  --for=condition=Ready \
  --all \
  -n falco \
  --timeout=180s
```

The Falco pod status was then verified using:

```
kubectl get pods -n falco
```

The Falco pod successfully entered the:

```
Running
```

state.

Evidence

```
osama@osama-VirtualBox:~/kubernetes-container-security-lab$ kubectl get pods -n falco

NAME          READY   STATUS    RESTARTS   AGE
falco-8zprj   2/2     Running   0          2m1s
```

Falco Running

7.2 Falco Runtime Environment

Falco startup logs confirmed that runtime monitoring was active.

The deployment used:

- Falco 0.44.1
- Modern BPF probe
- containerd container runtime
- Syscall event monitoring
- Kubernetes container metadata

This confirmed that Falco was capable of monitoring runtime activity occurring inside containers within the Minikube environment.

7.3 Initial `/etc` Write Test

The original project test attempted to generate a Falco event by writing a temporary file below `/etc`:

```
kubectl exec deployment/nginx-lab -- \
  sh -c 'touch /etc/test_file_for_falco_rule'
```

Falco logs were inspected using:

```
kubectl logs -n falco \
  -l app.kubernetes.io/name=falco \
  -c falco \
  --since=10m | grep Warning
```

In the installed Falco version and default ruleset, this activity did **not** generate a warning.

This result was documented rather than being treated as a successful detection.

The project guide specifically allows differences between Falco versions and recommends recording situations where a default rule does not trigger.

7.4 Successful Sensitive File Detection

To confirm that Falco runtime monitoring was functioning, another controlled event was generated from inside the Nginx container:

```
kubectl exec -it deployment/nginx-lab -- cat /etc/shadow
```

Falco generated a runtime warning indicating that a sensitive file had been opened for reading.

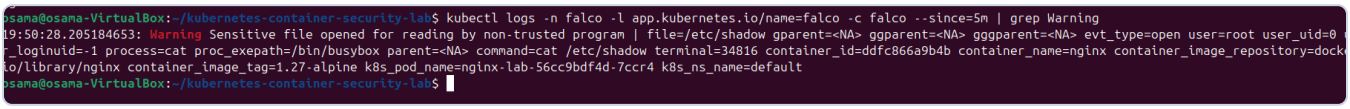
The alert contained information including:

```
Warning Sensitive file opened for reading by non-trusted program

file=/etc/shadow
user=root
process=cat
command=cat /etc/shadow
container_name=nginx
container_image_tag=1.27-alpine
k8s_ns_name=default
```

Falco also recorded the Kubernetes pod name and container identifiers.

Evidence



Falco Runtime Alert

7.5 Security Significance

The alert demonstrated that Falco could observe suspicious behavior occurring during container runtime rather than relying only on static Kubernetes configuration.

The detection provided useful investigation context, including:

- The affected file
- Executing user
- Process name
- Command line
- Container identity
- Container image

- Kubernetes pod
- Kubernetes namespace

This type of information could assist a security analyst in investigating suspicious activity inside a containerized environment.

7.6 Falco Monitoring Result

The Falco deployment was successfully installed and verified.

Although the original `/etc` file-write action was not detected by the default ruleset, access to `/etc/shadow` produced a clear runtime warning.

This confirmed that:

- Falco was operating successfully.
- Runtime system-call activity was being monitored.
- Container and Kubernetes metadata were available in alerts.
- Default detection coverage varied depending on the activity and enabled rules.

The `/etc` write detection gap was later addressed by creating and testing a custom Falco rule.

---

8. Controlled Security Testing

Three controlled security tests were performed inside the local Kubernetes lab.

The purpose of these tests was to generate repeatable evidence of suspicious container activity and insecure Kubernetes configuration.

The three tests defined by the project guide were:

1. Interactive shell access inside Nginx.
2. Writing a file below `/etc`.
3. Deploying a privileged pod.

All testing was performed only inside the self-owned local lab environment.

8.1 Test 1 - Interactive Shell Inside Nginx

Objective

Determine whether opening an interactive shell inside the running Nginx container would generate a Falco runtime alert.

The test command was:

```
kubectl exec -it deployment/nginx-lab -- /bin/sh
```

Once inside the container, the active user was checked using:

```
whoami
```

The shell was then exited.

Security Risk

Interactive container access can be security-sensitive because an attacker or unauthorized administrator with Kubernetes `exec` permissions could execute commands directly inside a production container.

Falco Result

Not Detected by the default ruleset

No corresponding warning was observed in the Falco logs during this test.

This does not mean that the activity is harmless. It demonstrates that detection depends on the enabled Falco rules.

Recommended Mitigations

Potential controls include:

- Restrict Kubernetes RBAC permissions for `pods/exec`.
- Run containers as non-root users.
- Disable privilege escalation.
- Drop unnecessary Linux capabilities.
- Create runtime-detection rules for unexpected shell execution.

Evidence

```
osama@osama-VirtualBox:~/kubernetes-container-security-lab$ kubectl exec -it deployment/nginx-lab -- /bin/sh
/ # whoami
root
/ # exit
osama@osama-VirtualBox:~/kubernetes-container-security-lab$
```

Interactive Container Shell

Detailed report:

```
reports/test-01-container-shell.md
```

8.2 Test 2 - File Written Below `/etc`

Objective

Determine whether writing a file below the `/etc` directory would generate a Falco warning.

The test command was:

```
kubectl exec deployment/nginx-lab -- \
  sh -c 'touch /etc/test_file_for_falco_rule'
```

Unexpected changes under `/etc` can be significant because they may indicate modification of system or application configuration files. The guide specifically identifies this as one of the controlled runtime tests.

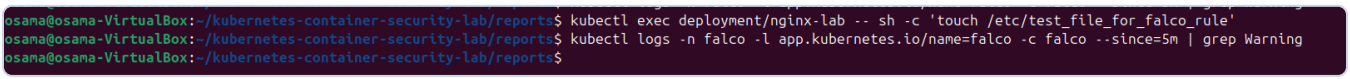
Initial Falco Result

Not Detected by the default ruleset

The command executed successfully, but the installed default Falco rules did not generate a corresponding warning.

This detection gap was documented rather than incorrectly reporting the test as detected.

Evidence



File Write Below Etc

Detailed report:

```
reports/test-02-file-write-etc.md
```

This test was later repeated after implementing a custom Falco detection rule.

8.3 Test 3 - Privileged Pod Deployment

Objective

Demonstrate the security risk associated with deploying a privileged Kubernetes container and determine whether Falco generated a runtime warning for the deployment.

The existing insecure pod was removed:

```
kubectl delete pod insecure-pod --ignore-not-found
```

It was then redeployed:

```
kubectl apply -f manifests/insecure-pod.yaml
```

The pod status was checked using:

```
kubectl get pod insecure-pod
```

The pod successfully entered the:

```
Running
```

state.

Security Risk

The workload contained:

```
securityContext:
  privileged: true
```

Privileged containers weaken normal container isolation and provide substantially more access to the host environment.

Falco Result

Not Detected by the default ruleset

No corresponding Falco warning was observed when the privileged pod was deployed.

The project therefore documented this primarily as a Kubernetes configuration-security finding.

The guide explicitly allows this result and states that if no default alert appears for the privileged pod, it should be documented as a configuration finding.

Evidence

```
osama@osama-VirtualBox:~/kubernetes-container-security-lab/manifests$ kubectl apply -f insecure-pod.yaml
pod/insecure-pod created
osama@osama-VirtualBox:~/kubernetes-container-security-lab/manifests$ kubectl get pod insecure-pod
NAME           READY   STATUS    RESTARTS   AGE
insecure-pod   1/1     Running   0          6s
osama@osama-VirtualBox:~/kubernetes-container-security-lab/manifests$
```

Privileged Pod Deployment

Detailed report:

```
reports/test-03-privileged-pod.md
```

8.4 Initial Test Summary

Test	Activity	Default Falco Result
Test 1	Interactive shell inside Nginx	Not Detected
Test 2	Write file below `/etc`	Not Detected
Test 3	Deploy privileged pod	Not Detected
Additional runtime verification	Read `/etc/shadow`	Detected

The three required controlled test commands were successfully attempted and individually documented.

Although the default ruleset did not alert on the three Day 5 activities in this Falco installation, the successful `/etc/shadow` detection had already confirmed that Falco runtime monitoring was operating.

The project guide notes that the lab can still be considered successful when some tests do not trigger, provided the results are recorded honestly and at least one clear runtime alert is captured.

The `/etc` write detection gap was subsequently used as an opportunity to implement a custom Falco rule and demonstrate detection engineering.

---

9. Custom Falco Detection Rule

During the initial testing phase, writing a file below `/etc` did not generate an alert using the default Falco ruleset.

Rather than leaving this as an unresolved detection gap, the project was extended by creating a custom Falco rule specifically designed to detect file-write activity below `/etc`.

This extension demonstrates basic detection-engineering skills in addition to using Falco as an off-the-shelf runtime monitoring tool.

9.1 Detection Gap Identified

The original test command was:

```
kubectl exec deployment/nginx-lab -- \
  touch /etc/test_file_for_falco_rule
```

The action completed successfully.

However, the default Falco rules did not generate a warning.

Initial Result

```
Not Detected
```

This established a clear detection gap.

9.2 Custom Rule Creation

A custom Falco configuration file was created:

```
falco/falco-custom-rules.yaml
```

The rule was named:

```
Write below etc
```

The custom rule was configured as follows:

```
customRules:
  custom-rules.yaml: |-
    - rule: Write below etc
      desc: Detect attempts to open files below /etc for writing
      condition: >
        (evt.type in (open,openat,openat2) and evt.is_open_write=true and fd.typechar='f' and
fd.num>=0)
        and fd.name startswith /etc
      output: >
        File below /etc opened for writing |
        file=%fd.name
        user=%user.name
        process=%proc.name
        command=%proc.cmdline
        container=%container.name
        image=%container.image.repository
        pod=%k8s.pod.name
        namespace=%k8s.ns.name
      priority: WARNING
      tags: [filesystem, mitre_persistence]
```

The purpose of the rule was to detect write-open operations targeting files underneath:

```
/etc
```

The output was configured to provide useful security investigation context.

9.3 Custom Rule Deployment

The existing Falco Helm release was upgraded using:

```
helm upgrade falco falcosecurity/falco \
  --namespace falco \
  --reuse-values \
  -f falco/falco-custom-rules.yaml
```

The Helm release was successfully upgraded.

Falco remained in the:

```
Running
```

state after the update.

The Helm values were inspected to confirm that the custom rule configuration was present.

Falco logs also confirmed successful schema validation:

```
/etc/falco/rules.d/custom-rules.yaml | schema validation: ok
```

This confirmed that Falco had accepted the custom rule configuration.

9.4 Retesting the Original Activity

To produce a clean retest, the previous test file was removed:

```
kubectl exec deployment/nginx-lab -- \
  rm -f /etc/test_file_for_falco_rule
```

The same original activity was then repeated:

```
kubectl exec deployment/nginx-lab -- \
  touch /etc/test_file_for_falco_rule
```

Falco logs were inspected using:

```
kubectl logs -n falco \
  -l app.kubernetes.io/name=falco \
  -c falco \
  --since=1m | grep "File below /etc"
```

9.5 Successful Custom Detection

This time, Falco generated the following warning:

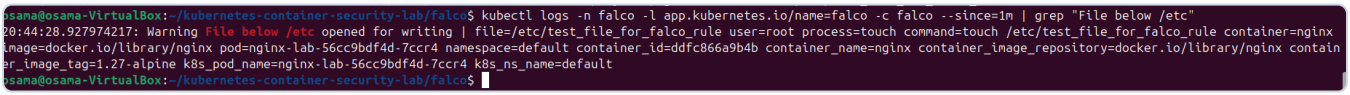
```
Warning File below /etc opened for writing |
file=/etc/test_file_for_falco_rule
user=root
process=touch
command=touch /etc/test_file_for_falco_rule
container=nginx
image=docker.io/library/nginx
pod=nginx-lab-56cc9bdf4d-7ccr4
namespace=default
```

Falco also recorded additional information including:

- Container ID
- Container name
- Container image repository
- Container image tag

- Kubernetes pod name
- Kubernetes namespace

Evidence



Custom Falco Rule Detection

Detailed report:

```
reports/test-04-custom-falco-rule.md
```

9.6 Before and After Detection Comparison

Stage	Activity	Detection Result
Default Falco configuration	Write file below `/etc`	Not Detected
Custom Falco rule enabled	Write file below `/etc`	**Detected**

This provided a direct before-and-after demonstration using the same activity.

The improvement showed that runtime monitoring can be adapted when the default ruleset does not cover a specific behavior.

9.7 Detection Engineering Result

The custom-rule extension demonstrated the following workflow:

Identify Detection Gap → Design Rule → Deploy Rule → Validate Rule → Repeat Activity → Confirm Detection

This is significant because the project no longer demonstrates only the installation and use of Falco.

It also demonstrates the ability to:

- Identify missing detection coverage.
- Understand the behavior that should be monitored.
- Create a targeted detection rule.
- Deploy the rule through Helm.

- Validate the configuration.
- Retest the same behavior.
- Interpret the resulting security alert.
- Document the before-and-after outcome.

The custom detection therefore strengthened the project from a basic runtime-monitoring lab into a more complete defensive-security and detection-engineering exercise.

---

10. Kubernetes Workload Hardening

After documenting the insecure configuration and completing the runtime-security tests, a hardened Kubernetes workload was created.

The hardened pod was designed to reduce container privileges, restrict filesystem access, and control resource consumption.

The manifest is stored in:

```
manifests/hardened-pod.yaml
```

The workload uses:

```
nginxinc/nginx-unprivileged:alpine
```

10.1 Non-Root Container Execution

The hardened container was configured to run as a non-root user:

```
runAsNonRoot: true
runAsUser: 101
runAsGroup: 101
```

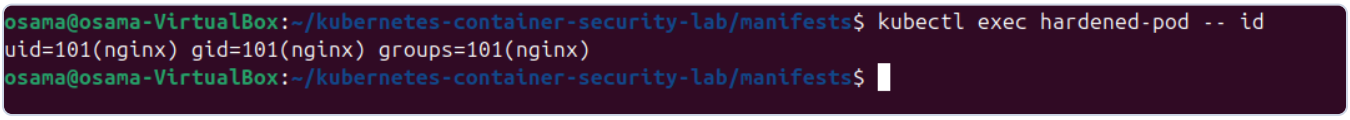
The runtime identity was verified using:

```
kubectl exec hardened-pod -- id
```

The result confirmed that the container was running with a non-zero UID rather than:

```
uid=0(root)
```

Evidence



Hardened Pod Non Root

This reduces the level of privilege available to a process if the container is compromised.

10.2 Privilege Escalation Disabled

The hardened security context included:

```
allowPrivilegeEscalation: false
```

This prevents processes inside the container from gaining additional privileges through mechanisms such as setuid binaries.

10.3 Linux Capabilities Dropped

The hardened container was configured to drop all Linux capabilities:

```
capabilities:
  drop:
    - ALL
```

Linux capabilities divide traditional root privileges into smaller individual permissions.

Dropping unnecessary capabilities reduces the actions available to a compromised process.

10.4 Read-Only Root Filesystem

The hardened workload also used:

```
readOnlyRootFilesystem: true
```

This prevents processes from modifying the main container filesystem.

A read-only filesystem can reduce opportunities for:

- Unauthorized configuration modification
- Malware persistence
- Replacement of application files
- Modification of system files

10.5 Read-Only Filesystem Troubleshooting

During the initial hardened pod deployment, the container failed to start correctly.

The Nginx logs showed:

```
mkdir() "/tmp/proxy_temp" failed (30: Read-only file system)
```

Nginx required writable temporary storage even though the main container filesystem had intentionally been configured as read-only.

One possible solution would have been to remove:

```
readOnlyRootFilesystem: true
```

However, this would have weakened the intended security configuration.

Instead, a dedicated temporary Kubernetes volume was created:

```
volumeMounts:
  - name: nginx-tmp
    mountPath: /tmp

volumes:
  - name: nginx-tmp
    emptyDir: {}
```

The `/tmp` directory therefore remained writable while the rest of the root filesystem remained read-only.

After applying this modification, the hardened pod successfully entered the:

```
Running
```

state.

This troubleshooting step demonstrated how application requirements can be supported without unnecessarily removing security controls.

10.6 Resource Requests and Limits

CPU and memory controls were introduced:

```
resources:
  requests:
    cpu: "50m"
    memory: "64Mi"
  limits:
    cpu: "100m"
    memory: "128Mi"
```

Resource requests specify the resources Kubernetes should reserve for the workload.

Resource limits restrict the maximum amount of CPU and memory that the container should consume.

These controls reduce the risk of a workload consuming excessive node resources.

10.7 Security Context Verification

The main security settings were verified using:

```
kubectl get pod hardened-pod -o yaml | \
grep -E "runAsNonRoot|runAsUser|runAsGroup|allowPrivilegeEscalation|readOnlyRootFilesystem|drop:|
cpu:|memory:"
```

Evidence

Hardened Security Context

The verification confirmed the presence of:

- Non-root execution
- UID and GID 101
- Disabled privilege escalation
- Read-only root filesystem
- Dropped capabilities
- CPU controls

- Memory controls

11. Default-Deny NetworkPolicy

A Kubernetes NetworkPolicy was introduced to demonstrate basic network isolation.

The policy is stored in:

```
manifests/default-deny-networkpolicy.yaml
```

The configuration was:

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: default-deny
spec:
  podSelector: {}
  policyTypes:
    - Ingress
    - Egress
```

The policy was applied using:

```
kubectl apply -f manifests/default-deny-networkpolicy.yaml
```

It was verified using:

```
kubectl get networkpolicy
```

The cluster returned:

```
NAME           POD-SELECTOR
default-deny   <none>
```

Further details were inspected using:

```
kubectl describe networkpolicy default-deny
```

The policy showed:

```
Allowing ingress traffic:
  <none>

Allowing egress traffic:
  <none>

Policy Types: Ingress, Egress
```

The empty pod selector:

```
podSelector: {}
```

selects all pods in the namespace for the policy.

The absence of ingress and egress allow rules represents a default-deny configuration for selected pods.

Evidence

```
osama@osama-VirtualBox:~/kubernetes-container-security-lab/manifests$ kubectl apply -f default-deny-networkpolicy.yaml
networkpolicy.networking.k8s.io/default-deny created
osama@osama-VirtualBox:~/kubernetes-container-security-lab/manifests$ kubectl get networkpolicy
NAME           POD-SELECTOR   AGE
default-deny   <none>         13s
osama@osama-VirtualBox:~/kubernetes-container-security-lab/manifests$ kubectl describe networkpolicy default-deny
Name:         default-deny
Namespace:    default
Created on:   2026-08-10 23:17:05 +0300 +03
Labels:       <none>
Annotations:  <none>
Spec:
  PodSelector:     <none> (Allowing the specific traffic to all pods in this namespace)
  Allowing ingress traffic:
    <none> (Selected pods are isolated for ingress connectivity)
  Allowing egress traffic:
    <none> (Selected pods are isolated for egress connectivity)
  Policy Types: Ingress, Egress
```

Default Deny NetworkPolicy

NetworkPolicy enforcement depends on support from the Kubernetes networking implementation, which is an important limitation of the local lab environment.

11.1 Before and After Hardening Comparison

Security Control	Insecure Workload	Hardened Workload
Container user	Root (`uid=0`)	Non-root (`uid=101`)
Privileged mode	Enabled	Not enabled
Privilege escalation	Not restricted	Disabled

Security Control	Insecure Workload	Hardened Workload
Linux capabilities	Default capabilities	All dropped
Root filesystem	Writable	Read-only
Writable temporary storage	General filesystem	Dedicated `/tmp` volume
CPU requests/limits	None	Configured
Memory requests/limits	None	Configured
NetworkPolicy	None	Default deny
Runtime monitoring	Initially absent	Falco enabled

11.2 Hardening Result

The hardened workload successfully introduced multiple defense-in-depth controls.

The final configuration:

- Runs as a non-root user.
- Prevents privilege escalation.
- Drops Linux capabilities.
- Uses a read-only root filesystem.
- Provides only the required writable temporary storage.
- Defines CPU requests and limits.
- Defines memory requests and limits.
- Uses a default-deny NetworkPolicy.

The hardened workload therefore provides a clear contrast with the intentionally insecure baseline and demonstrates practical Kubernetes container-security controls.

12. Final Project Results

The project successfully demonstrated the full lifecycle of a Kubernetes container-security exercise.

The completed workflow was:

Deploy → Identify → Test → Detect → Harden → Retest → Document

Final Results Summary

Area	Result
Local Kubernetes cluster	Successfully deployed using Minikube
Nginx application	Successfully deployed and exposed
Insecure workload	Root and privileged configuration demonstrated
Security weaknesses	Root execution, privileged mode, no limits, and no initial NetworkPolicy identified
Falco	Successfully installed and operational
Runtime detection	Sensitive `/etc/shadow` access detected
Controlled Test 1	Interactive shell not detected by default rules
Controlled Test 2	`/etc` file write not detected by default rules
Controlled Test 3	Privileged pod deployment not detected by default rules
Custom Falco rule	Successfully created and deployed
`/etc` write retest	Successfully detected after custom rule
Hardened workload	Successfully deployed
Non-root execution	Verified
Privilege escalation	Disabled
Linux capabilities	All dropped
Read-only root filesystem	Enabled
Resource controls	CPU and memory limits configured
NetworkPolicy	Default-deny policy created

Area	Result
Documentation	YAML, reports, screenshots, README, and architecture diagram completed

13. Key Lessons Learned

Several important technical and security lessons were demonstrated during the project.

13.1 Security Requires Multiple Layers

No single security control was sufficient by itself.

The project combined:

- Secure container configuration
- Runtime monitoring
- Resource controls
- Network restrictions
- Detection rules
- Documentation and verification

This demonstrates the concept of defense in depth.

13.2 Default Detection Rules Do Not Detect Everything

Falco successfully detected access to `/etc/shadow`, but several other controlled activities were not detected by the default ruleset.

This demonstrated that installing a security-monitoring platform does not automatically provide complete detection coverage.

Detection logic must be adapted to the environment and the behaviors that an organization considers important.

13.3 Detection Gaps Can Be Improved

The `/etc` file-write test initially produced no alert.

A custom Falco rule was then created and deployed.

After repeating the same activity, Falco successfully generated an alert.

This demonstrated a basic detection-engineering workflow:

```
Identify Gap
    ↓
Create Detection
    ↓
Deploy Rule
    ↓
Validate
    ↓
Retest
    ↓
Confirm Alert
```

13.4 Security Controls Can Affect Application Functionality

Enabling:

```
readOnlyRootFilesystem: true
```

initially prevented Nginx from operating because it required writable temporary storage.

Rather than removing the security control, only the required `/tmp` directory was made writable using an `emptyDir` volume.

This demonstrated the importance of understanding both application requirements and security controls when hardening workloads.

---

14. Project Limitations

Although the lab successfully demonstrated Kubernetes security concepts, it has several limitations.

14.1 Single-Node Kubernetes Environment

The project used Minikube with one Kubernetes node.

Production Kubernetes environments normally contain multiple worker nodes and more complex infrastructure.

14.2 Local Virtualized Environment

The lab was hosted inside VirtualBox rather than a production cloud or enterprise Kubernetes platform.

The environment therefore does not reproduce all networking, identity, availability, and infrastructure-security considerations found in production systems.

14.3 Simple Application

Nginx was used as the primary application workload.

A real application may contain:

- Multiple containers
- Databases
- APIs
- Secrets
- Service accounts
- Persistent storage
- Multiple namespaces
- External services

These would create additional security considerations.

14.4 Falco Detection Coverage

The default Falco rules did not detect all controlled activities.

One custom detection rule was created to demonstrate how detection coverage could be extended, but a production environment would require a much larger and continuously maintained ruleset.

14.5 NetworkPolicy Validation

A default-deny NetworkPolicy was successfully created and inspected.

However, enforcement depends on the Kubernetes networking implementation.

A future version of the project could use a networking solution such as Calico and perform explicit connectivity tests before and after applying policies.

14.6 No Centralized SIEM

Falco alerts were inspected directly using Kubernetes logs.

The project did not forward alerts into a centralized SIEM platform for:

- Searching
- Correlation
- Dashboards
- Alert triage
- Long-term storage
- Incident investigation

---

15. Future Improvements

The project could be extended in several directions.

15.1 Falco and Wazuh Integration

Falco alerts could be forwarded into Wazuh.

This would allow runtime Kubernetes alerts to be:

- Centralized
- Indexed
- Searched
- Correlated
- Displayed in dashboards
- Used during SOC-style investigations

15.2 Additional Custom Falco Rules

More custom rules could be developed for behaviors such as:

- Unexpected interactive shells
- Suspicious process execution
- Sensitive configuration changes
- Package-manager execution
- Unexpected network tools
- Privileged container activity

Each rule could follow the same:

Before → Detection Rule → Retest → After

methodology used in this project.

15.3 Kubernetes Audit Logging

Kubernetes audit logs could be enabled to monitor Kubernetes API activity.

This would provide visibility into actions such as:

- Pod creation
- `kubectl exec`

- Secret access
- Role changes
- Deployment modification
- Administrative actions

15.4 Pod Security Admission

Pod Security Admission could be introduced to prevent insecure workloads from being deployed rather than only identifying them afterward.

Policies could restrict behaviors including:

- Privileged containers
- Root execution
- Dangerous capabilities
- Host namespace access

15.5 Advanced NetworkPolicy Testing

A compatible network plugin such as Calico could be deployed.

Connectivity tests could then demonstrate:

```
Before NetworkPolicy → Traffic Allowed

After Default Deny → Traffic Blocked

After Explicit Allow Rule → Required Traffic Allowed
```

15.6 Cloud Kubernetes Deployment

The lab could eventually be recreated on a managed Kubernetes platform such as:

- Azure Kubernetes Service
- Amazon Elastic Kubernetes Service
- Google Kubernetes Engine

This would introduce additional areas including:

- Cloud IAM
- Managed Kubernetes security
- Cloud networking

- Load balancers
- Cloud logging
- Production-style infrastructure

15.7 CI/CD Security

Future development could integrate security checks into a CI/CD pipeline.

Possible controls include:

- Kubernetes manifest scanning
- Container image scanning
- Secret scanning
- Dependency scanning
- Automated policy validation

This would extend the project from runtime security into DevSecOps.

---

16. Project Conclusion

The Kubernetes Container Security and Runtime Detection Lab successfully demonstrated practical container and Kubernetes security concepts within a controlled local environment.

The project began by creating a functional Kubernetes cluster and deploying an Nginx application.

An intentionally insecure workload was then introduced and analyzed. The workload demonstrated several security weaknesses, including root execution, privileged mode, missing resource controls, and the absence of an initial NetworkPolicy.

Falco was installed to provide runtime-security monitoring. A sensitive-file access test successfully produced a runtime alert containing container, process, user, and Kubernetes context.

Three controlled security tests were then conducted. Although the installed default Falco rules did not detect each test, these results were documented accurately.

The missing `/etc` write detection was subsequently treated as a detection-engineering problem. A custom Falco rule was created, deployed, validated, and tested. Repeating the same activity successfully generated an alert, providing a clear before-and-after detection improvement.

The insecure Kubernetes configuration was then replaced with a hardened workload using:

- Non-root execution
- Disabled privilege escalation
- Dropped Linux capabilities
- A read-only root filesystem
- Restricted writable temporary storage
- CPU and memory controls
- A default-deny NetworkPolicy

During the hardening process, an application compatibility problem caused by the read-only filesystem was diagnosed and resolved without removing the intended security control.

Overall, the project demonstrates practical experience in:

- Kubernetes
- Containers
- Linux
- Runtime security monitoring
- Falco
- Detection engineering

- Container hardening
- Network security
- Troubleshooting
- Git and GitHub
- Security documentation

The final result is a reproducible cybersecurity portfolio project demonstrating both preventive and detective Kubernetes security controls.

---

17. Safety and Authorization

All security testing performed during this project was conducted only inside a self-owned local lab environment.

No systems belonging to third parties were targeted.

The intentionally insecure configurations and controlled security tests were created exclusively for educational, cybersecurity-training, and portfolio purposes.

18. Final Evidence

The complete project repository contains:

```
manifests/
    Kubernetes workload and security configurations

falco/
    Falco notes and custom detection rules

reports/
    Security test and hardening reports

screenshots/
    Runtime and configuration evidence

diagrams/
    Architecture diagram

docs/
    Final project report
```

The repository therefore contains both the technical implementation and the evidence required to reproduce and explain the project.